Industrial Internship Report on

“Smart Agriculture Monitoring System”

An ESP32-based IoT Sensor Node for Automated Soil, Climate and Light Monitoring

Prepared by

Dulam Shiva

JBIET

Internship Duration: 15 September 2026 – 27 October 2026 (6 weeks)

Industrial Partner: UniConverge Technologies Pvt Ltd (UCT)
In association with upskill Campus (USC) and The IoT Academy

Executive Summary

This report presents the work carried out during a 6-week Industrial Internship (15 September – 27 October 2026) provided by upskill Campus (USC) and The IoT Academy, in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). The internship was centred on a problem statement provided by UCT, to be completed, along with this report, within the 6-week period.

My project was a Smart Agriculture Monitoring System: an ESP32-based IoT sensor node that continuously measures soil moisture, ambient temperature and humidity, and light intensity, displays live status on an onboard LCD, timestamps every reading using a real-time clock, and automatically drives a water pump via a relay when the soil is classified as dry. The system was built and verified using the Wokwi circuit simulator, which allowed the full sensor-to-actuator pipeline to be developed, debugged and demonstrated without needing physical hardware.

Beyond the circuit and firmware itself, the internship involved tracing and correcting several real firmware defects — an ADC scaling error that produced numerically wrong light readings, a duplicated Serial initialization at conflicting baud rates, and a large, partially broken lookup table standing in for what should have been a simple linear calculation. This internship gave me a very good opportunity to get exposure to the kind of debugging and design discipline expected in industrial embedded systems work, well beyond what a purely academic exercise would have required. It was, overall, a valuable and challenging experience.

1. Preface

Over the course of this 6-week internship, I moved from a surface-level understanding of ESP32-based IoT systems to actually building, debugging and documenting a complete sensor node from scratch. The work was not simply a matter of assembling a circuit and uploading example code — it required understanding why each sensor behaves the way it does, how the ESP32's ADC and I2C peripherals actually work, and how to track a symptom (a sensor “read failure”, or numbers that look plausible but are wrong) back to its actual root cause rather than treating a simulator's silence as proof of correctness.

An internship of this kind is a necessary complement to classroom learning in early-career technical education. Lecture-based study teaches the underlying theory — how an ADC quantizes a voltage, how I2C arbitration works — but it rarely forces the kind of disciplined, evidence-based debugging that real hardware and real deadlines demand. This internship did exactly that, under the structure and mentorship made possible by USC and UCT.

My project, assigned as part of this program, was to design, build and document a Smart Agriculture Monitoring System capable of autonomously tracking a farm plot's key environmental parameters and triggering irrigation without manual intervention. The program was structured as a self-paced 6-week build-and-document cycle, with the expectation of a working simulation, a documented and corrected codebase, and this report as the final deliverables.

I am grateful to upskill Campus, The IoT Academy, and UniConverge Technologies Pvt Ltd for structuring and making this opportunity available. To my juniors and peers approaching a similar internship: treat every “it works” moment with suspicion until you've checked why it works — a simulator, in particular, will happily run code with real bugs in it (wrong ADC scaling, logically dead branches) without ever raising an error. The habit of verifying your own assumptions is the single most useful thing I am taking from this program.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in the Digital Transformation domain, providing Industrial solutions with a prime focus on sustainability and RoI. For developing its products and solutions, it leverages various cutting-edge technologies, e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, and front-end development.

UCT Insight is an IoT platform designed for quick deployment of IoT applications while providing valuable "insight" for a business process. It is built in Java for the backend and ReactJS for the front end, with support for MySQL and various NoSQL databases. It enables device connectivity via industry-standard IoT protocols — MQTT, CoAP, HTTP, Modbus TCP, OPC UA — and supports both cloud and on-premises deployments, with dashboarding, analytics and reporting, alerting, third-party integration (Power BI, SAP, ERP) and a rule engine.

UCT also offers a Smart Factory Platform for production and asset monitoring, OEE and predictive maintenance up to digital-twin scale; a LoRaWAN-based solution portfolio for Agritech, smart cities, industrial monitoring, smart street lighting and smart utility metering; and Predictive Maintenance solutions that combine embedded systems, industrial IoT and machine learning to estimate remaining useful life for production machinery.

2.2 About upskill Campus (USC)

upskill Campus, along with The IoT Academy and in association with UniConverge Technologies, facilitated the smooth execution of this internship. USC is a career development platform delivering personalized, self-paced upskilling together with supporting services such as internships, projects, interaction with industry experts, and career growth services (upskillcampus.com). USC's stated aim is to upskill 1 million learners over the next 5 years.

2.3 The IoT Academy

The IoT Academy is the EdTech division of UCT, running long-form executive certification programs in collaboration with EICT Academy, IIT Kanpur, IIT Roorkee and IIT Guwahati, across multiple domains.

2.4 Objective

The objectives of this internship program were to:

- Gain practical experience of working in an industrial setting.
- Solve a real-world problem, not a purely academic exercise.
- Improve job prospects through demonstrable, hands-on project work.
- Build a deeper understanding of embedded systems and IoT, and their real applications.
- Develop personal skills such as communication, documentation and structured problem-solving.

2.5 Reference

[1] Espressif Systems — ESP32 Technical Reference Manual

[2] Adafruit — DHT22 Temperature & Humidity Sensor

[3] Analog Devices / Maxim — DS1307 64 x 8, Serial, I2C Real-Time Clock Datasheet

[4] Wokwi — Simulator Documentation

2.6 Glossary

Term	Meaning
ESP32	A dual-core Wi-Fi/Bluetooth-capable microcontroller (MCU) used as the system's main controller.
ADC	Analog-to-Digital Converter — converts an analog sensor voltage into a digital value the MCU can process.
I2C	Inter-Integrated Circuit — a two-wire (SDA/SCL) serial bus used here to connect the LCD and RTC.
RTC	Real-Time Clock — a dedicated IC (DS1307) that keeps accurate time independent of the MCU.
LCD	Liquid Crystal Display — used here (20×4, I2C-driven) to show live sensor status.
GPIO	General Purpose Input/Output — digital pins used to read/drive sensors, LEDs and the relay.
FC-28	A resistive soil moisture sensor whose output voltage varies with soil moisture content.
DHT22	A digital sensor that reports temperature and relative humidity over a single-wire protocol.

3. Problem Statement

Design and implement an IoT-based Smart Agriculture Monitoring System capable of continuously tracking the key environmental parameters that determine crop health — soil moisture, ambient temperature and humidity, and light level — and of acting on that data automatically, without requiring a farmer to manually check conditions and operate irrigation equipment.

The assigned problem statement required the system to:

- Read soil moisture, temperature, humidity and light level from appropriate sensors at a regular interval.
- Classify each reading into an actionable state (e.g. soil Wet / Normal / Dry; light Low / Medium / High).
- Display the current status locally, for a user standing at the device, with a timestamp.
- Automatically control a water pump based on the soil moisture classification, without manual intervention.
- Be buildable and demonstrable without requiring access to a physical farm plot or costly field hardware during development.

4. Existing and Proposed Solution

4.1 Existing Solutions and Their Limitations

Traditional irrigation in small and mid-sized farms largely relies on fixed schedules or manual inspection — a farmer checks soil by hand and waters on a routine, independent of actual need. This wastes water on already-adequate soil and under-waters during unexpected dry spells. Commercial “smart farm” systems exist, but they are typically priced and architected for large commercial operations: proprietary sensor networks, cloud subscriptions, and installation costs that put them out of reach for a small farm or an individual plot. Simpler consumer soil sensors (the kind sold as

standalone moisture meters) give a single manual reading and have no automation, logging or display capability at all.

4.2 Proposed Solution

The proposed solution is a low-cost, self-contained ESP32-based sensor node that performs continuous, automated monitoring and irrigation control locally, with no dependency on a commercial cloud platform or recurring subscription. It combines commodity sensors (DHT22, an FC-28-type soil probe, an LDR light sensor, a DS1307 RTC for timestamping) with a 20×4 I2C LCD for an at-a-glance local display and a relay-driven pump for automatic irrigation — all on a single ESP32 board costing a small fraction of a commercial system.

4.3 Value Addition

- Fully local operation: the system monitors and acts without depending on internet connectivity or a cloud service, important for rural sites with unreliable network coverage.
- Automatic, threshold-based pump control removes the farmer from the irrigation decision loop entirely for routine conditions.
- Low component cost relative to commercial smart-agriculture platforms, since every part used is a commodity module.
- Built and verified in the Wokwi simulator first, allowing the full design to be developed, debugged and demonstrated before any physical hardware purchase — reducing both cost and risk during development.

4.4 PROJECT Submission

GitHub repository: [GitHub link here]

https://github.com/ShivaDulam-06/upskillcampus/tree/main/Smart%20Agriculture%20System%20%20IOT%20Project-USC_UCT

4.5 Report Submission

GitHub repository: [GitHub link here]

https://github.com/ShivaDulam-06/upskillcampus/blob/main/SmartAgricultureMonitoringSystem_DulamShiva_USC_UCT

5. Proposed Design / Model

5.1 High-Level Diagram

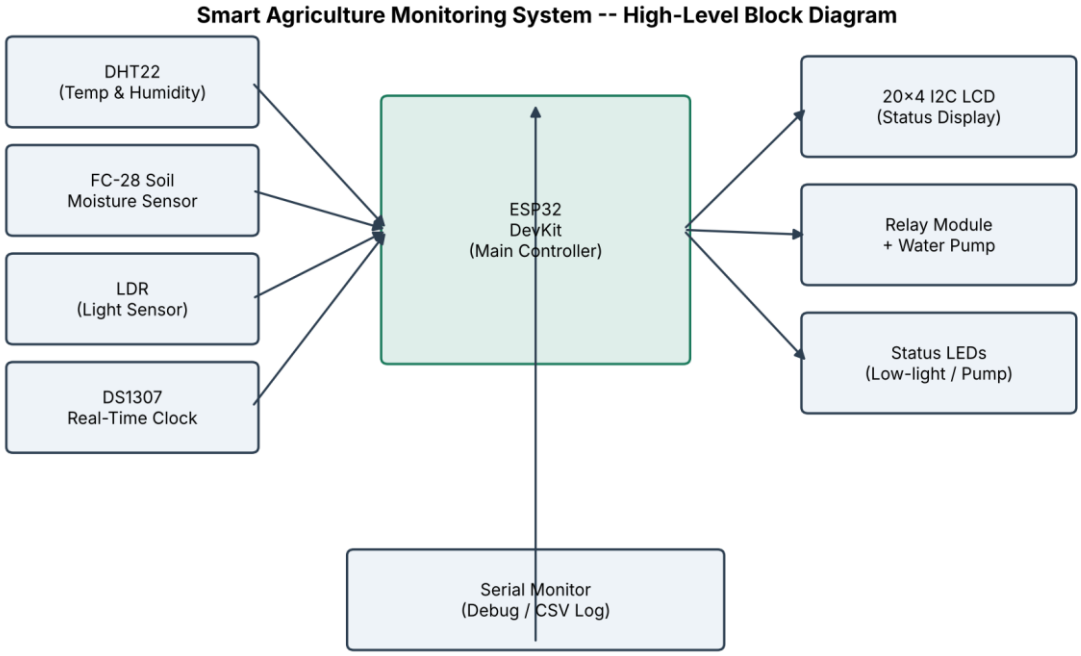


Figure 1: High-Level Diagram of the System

5.2 Low-Level Diagram — Pin Mapping

Component	Signal	ESP32 Pin
DHT22	Data	D15
FC-28 soil sensor (simulated via potentiometer in Wokwi)	Analog out	D34
LDR module	Analog out	D35
I2C bus (shared)	SDA / SCL	D21 / D22
20×4 LCD	I2C	D21 / D22 (shared)
DS1307 RTC	I2C	D21 / D22 (shared)
Relay (pump)	IN	D18
Low-light status LED	Anode (via 220Ω resistor)	D33

5.3 Interfaces

The system uses three interface types. A shared I2C bus (SDA = D21, SCL = D22) connects both the LCD and the DS1307 RTC, addressed independently at their own I2C addresses. Analog sensors (soil moisture, light) are read via the ESP32's ADC1 channels (D34, D35 — both input-only, ADC-capable pins, chosen specifically to avoid conflicts with pins used for other on-chip functions). Digital GPIO handles the DHT22's single-wire protocol (D15), the relay's control line (D18) and the status LED (D33, current-limited through a 220Ω resistor).

Data flow is a simple polling loop: every ~3 seconds, the controller reads all sensors, classifies each reading, updates the LCD and Serial log (timestamped via the RTC), and evaluates the pump-control condition. There is no interrupt-driven or event-based logic in this version — polling at a fixed interval was judged sufficient given the slow rate of change of the physical quantities being measured (soil moisture and ambient climate do not change on sub-second timescales).

6. Performance Test

This system was developed and verified entirely within the Wokwi circuit simulator. The constraints below reflect what could realistically be tested in that environment; constraints that depend on real physical behaviour (actual sensor accuracy, real power consumption, real ambient noise) are identified but could not be measured, and are called out explicitly rather than reported as tested.

6.1 Test Plan / Test Cases

ID	Test Case	Expected Result
TC1	DHT22 reports temperature/humidity within the simulator's configured range	Serial log and LCD display plausible, stable values each cycle
TC2	Soil sensor (potentiometer) swept from one extreme to the other	LCD “Soil” status transitions Wet → Normal → Dry as the dryness value crosses 33.33% and 66.66%
TC3	Soil dryness value exceeds 66.66%	Relay energizes (pump ON), LCD shows a “Dry” alert message
TC4	LDR value swept from bright to dark	LCD light status transitions High → Medium → Low; low-light LED switches ON below the Low threshold
TC5	RTC availability check at boot	On a missing/undetected RTC, the system halts and displays “RTC Not Connected” rather than continuing with invalid timestamps
TC6	Serial baud-rate consistency	A single, consistent baud rate (9600) is used throughout — no duplicate or conflicting Serial.begin() calls

6.2 Test Procedure

Each test case was executed by running the project in Wokwi, manipulating the relevant simulated input (the potentiometer for soil moisture, the DHT22's and LDR's configurable attributes for climate and light), and observing both the LCD output and the Serial monitor log for the expected transition or value. Firmware defects were identified by comparing observed behaviour against the expected physical relationship — for example, the LDR's calculated lux value was checked against the known ESP32 ADC characteristics (12-bit, ~3.3V reference) rather than accepted simply because the simulation produced a number without error.

6.3 Performance Outcome

All six test cases above passed after the following defects, found during testing, were corrected:

- ADC reference-voltage mismatch: the light-level calculation originally assumed a 10-bit/5V ADC (an Arduino Uno convention) instead of the ESP32's actual 12-bit/3.3V ADC, producing numerically incorrect lux values that nonetheless looked plausible on the LCD. Corrected to use the ESP32's real ADC characteristics.
- Duplicate, conflicting Serial initialization: the sensor driver (FC28Sensor) called Serial.begin() internally at 115200 baud, overriding main.ino's own Serial.begin(9600) call. Corrected by removing Serial control from the driver entirely — a sensor driver should never own global hardware state.
- A ~900-line hand-built lookup table for soil moisture percentage contained at least three transposed comparison bounds, making those exact values permanently unreachable. Replaced with the equivalent single-line linear calculation, verified to reproduce the table's own intended output exactly.

Not tested, and explicitly flagged as a limitation: real sensor accuracy and drift, actual power consumption, and physical signal integrity (pull-up resistor behaviour, bus capacitance, ESD, ground loops) — none of these are modelled by the Wokwi simulator and would require validation on physical hardware before any field deployment.

7. My Learnings

This internship was my first experience Create something, not just making it run. The most important lesson was that a simulator producing a plausible-looking number is not the same as that number being correct — the ADC scaling bug would have shipped unnoticed if I had only checked that the LCD displayed something, rather than checking the displayed value against the ESP32's actual hardware characteristics.

- How to read and reason about a microcontroller's actual ADC resolution and reference voltage, rather than assuming conventions carried over from a different board (Arduino Uno's 10-bit/5V ADC vs. the ESP32's 12-bit/3.3V ADC).
- Why a peripheral driver should never own global hardware state (such as calling Serial.begin()) — and how a subtle ordering bug like that can silently change a system's effective configuration.
- How to recognize dead/unreachable code paths in a large conditional ladder, and why a derived linear formula is both more correct and more maintainable than a hand-copied lookup table.
- The value of documenting a non-obvious design decision (such as the soil sensor's “higher value = drier” convention) directly in code comments, so the next person — or an examiner — does not mistake an intentional design choice for a bug.
- How to use a circuit simulator (Wokwi) as a legitimate development and verification tool, while staying honest about what it can and cannot prove about real-world behaviour.

These are directly applicable to my career goal of becoming a job-ready embedded/IoT engineer: they are the same debugging and design-review habits expected on a professional firmware team, not academic-only exercises.

8. Future Work Scope

- Wi-Fi and MQTT connectivity: an earlier version of this project connected to a Wi-Fi network and an MQTT broker for remote monitoring and pump control; reintroducing this would allow the system to be monitored and operated away from the device itself.
- Validation on physical hardware: replacing the simulated potentiometer with a real FC-28 probe and testing against real soil, to validate the moisture-to-percentage mapping and check for the signal-integrity issues a simulator cannot model (pull-up resistor sizing, cable length, ESD).
- Cloud or SD-card data logging: persisting sensor history for trend analysis, rather than only showing live/current values.
- Power-budget analysis and low-power (deep-sleep) operation, important for any version intended to run on battery or solar power in the field.
- Calibration of the light-level (lux) formula against a real lux meter, since the current calculation is a standard LDR voltage-divider approximation rather than a sensor-specific calibrated curve.

- A weatherproof enclosure and field-deployable power supply, required before any outdoor pilot deployment.